GUIDE DE RÉFÉRENCE · DFIR · MEMORY FORENSICS

Forensique de la mémoire vive .

Analyser une machine en vie — lire la RAM comme la scène de crime où l'attaque se déroule vraiment.

0xKuraishY

AUTEUR · ÉDITION & RECHERCHE

Édition 2026 · v1.0
Public : analyste SOC confirmé
Volatility 3 · méthode & détection



À propos de ce document

AVANT-PROPOS · MODE D'EMPLOI

Ce guide traite la mémoire vive comme un terrain d'enquête à part entière. Il s'adresse à l'analyste SOC qui sait déjà trier des alertes et lire un rapport EDR, et qui veut comprendre ce que la RAM révèle qu'aucune autre source ne donne : l'état réel d'une machine à un instant t.

CE QUE VOUS Y TROUVEREZ

D'abord le terrain — comment le système d'exploitation organise la mémoire. Ensuite l'acquisition propre et ce que lit réellement Volatility 3. Enfin la chasse : injection, *process hollowing*, processus et connexions cachés, hooks, secrets en clair, artefacts réseau volatils.

COMMENT LIRE LES FICHES

Chaque technique de détection suit le même gabarit : **idée clé**, **mécanisme**, **ce qu'on voit en mémoire**, **piège & anti-forensique**, puis un **À retenir** compact. Les noms de plugins sont donnés tels qu'ils s'écrivent en ligne de commande.

POSITION DE CE SUJET

La forensique mémoire n'est pas un substitut de l'analyse de binaire : elle en est le **complément naturel**. L'analyse de binaire répond à « de quoi ce fichier est-il capable ? ». La mémoire répond à « qu'est-ce qui s'est *réellement* exécuté sur cet hôte, y compris ce qui n'a jamais touché le disque ? ». Le dernier chapitre est entièrement consacré à cette articulation.

Convention : les développements visent Windows x86-64, de loin le plus fréquent en SOC, avec des renvois Linux lorsqu'ils éclairent le propos. Les commandes et noms de modules suivent la documentation stable de Volatility 3 (série v2.28) ; le framework évoluant vite, vérifiez toujours la disponibilité d'un plugin sur votre version. Les sources vérifiées sont listées en fin d'ouvrage.

© 2026 0xKuraishY — document pédagogique défensif. Toute acquisition mémoire doit être menée dans un cadre légal, sur des systèmes dont vous avez la responsabilité ou l'autorisation explicite.

Sommaire

PARCOURS DE LECTURE — DU TERRAIN À LA CHASSE

I — LE TERRAIN : COMMENT L'OS ORGANISE LA MÉMOIRE

01 Pourquoi la mémoire vive, et pourquoi maintenant 5

02 Adressage virtuel, pages et traduction d'adresses 7

03 Le processus vu par le noyau : EPROCESS, PEB, VAD, threads 9

04 Le royaume noyau : pools, objets et surface DKOM 11

II — ACQUÉRIR PROPREMENT ET LIRE LA RAM

05 Acquisition : ordre de volatilité, outils, intégrité, pièges 13

06 Ce que lit réellement Volatility 3 : couches, symboles, automagic 15

III — LA CHASSE : DÉTECTER EN MÉMOIRE

07 Injection de code en mémoire 19
08 Process hollowing (processus creux) 20
09 Processus et modules cachés 22
10 Connexions et artefacts réseau volatils 23
11 Hooks : détourner le flux et aveugler l'EDR 24
12 Identifiants et clés en clair 26

IV — MISE EN PERSPECTIVE

13 Mémoire vive & analyse de binaire : deux moitiés d'une enquête 29
14 Méthode, limites et anti-forensique 30
15 Aide-mémoire des plugins & sources vérifiées 31

PARTIE I

Le terrain : comment l'OS organise la mémoire

On ne lit pas une image mémoire « au hasard ». Chaque plugin repose sur des structures précises que le noyau tient à jour pour fonctionner. Comprendre l'adressage virtuel, la pagination, l'anatomie d'un processus et les pools noyau, c'est savoir où regarder — et pourquoi un attaquant va chercher à y mentir.

Pourquoi la mémoire vive, et pourquoi maintenant

CHAPITRE 01 · LE TERRAIN

La RAM est la seule source qui contient l'état d'exécution réel d'une machine : code déchiffré, clés en clair, connexions actives, processus qu'aucun outil disque ne verra. Elle disparaît à l'extinction. C'est à la fois sa valeur et son urgence.

1.1 La RAM contient ce que le disque n'a jamais vu

Un disque raconte ce qui a été écrit ; la mémoire raconte ce qui *tourne*. Cette distinction est devenue décisive. Les charges utiles modernes vivent de plus en plus en mémoire seule – injection réflective, PE sans fichier de support, assemblies .NET chargées à la volée, scripts interprétés en RAM – et ne laissent sur disque aucun artefact exploitable. Face à ce type d'activité, l'analyse disque arrive les mains vides là où la mémoire conserve l'intégralité du code exécuté.

Trois familles d'éléments n'existent, en pratique, que dans la RAM d'un système en marche :

- **Le code réellement exécuté.** Un échantillon empaqueté ou chiffré est opaque sur disque ; en mémoire il est déballé et déchiffré, puisqu'il faut bien que le processeur l'exécute. La mémoire livre la version « vraie » du code que l'analyse statique seule ne peut atteindre.
- **Les secrets transitoires.** Clés de chiffrement de volume (BitLocker, VeraCrypt) présentes tant que le volume est monté, mots de passe et *tokens* en clair, matériel cryptographique de session : autant d'éléments qui ne sont jamais censés être écrits sur disque.
- **L'état volatil du système.** Connexions réseau établies, sockets et ports en écoute, arborescence réelle des processus, handles ouverts, historique de commandes console – un instantané vivant qui s'évapore au redémarrage.

1.2 Volatilité : l'ordre dans lequel les preuves s'effacent

Toute collecte sur système actif obéit à un principe simple : capturer d'abord ce qui disparaît le plus vite. La hiérarchie classique, formalisée par la RFC 3227, va des registres et caches du processeur (nanosecondes) à la RAM (secondes à minutes), puis à l'état réseau, à l'état des processus, aux fichiers temporaires et au *pagefile*, enfin aux disques permanents et aux sauvegardes.

> CONSÉQUENCE PRATIQUE
>
> Sur une machine allumée et suspecte, **la mémoire se capture avant de toucher au disque**. Une imagerie disque qui sature les E/S provoque des évictions de cache, des changements d'état réseau et des échanges de pages qui détruisent précisément les preuves mémoire que l'on cherche à préserver.

1.3 « Machine en vie » : deux lectures, un même but

Analyser une machine en vie recouvre deux gestes qu'il faut distinguer. **L'analyse mémoire hors-ligne** – le cœur de ce guide – travaille sur une *image* acquise, stable et rejouable autant de fois que nécessaire. **L'analyse mémoire en direct**, pointée sur la mémoire physique d'un hôte en marche, existe mais reste l'exception : la mémoire bouge sous l'outil, et la plupart des frameworks modernes font le choix assumé de lire une image fi-

gée plutôt que de poursuivre une cible mouvante. Dans les deux cas, l'objectif est le même : reconstruire, à partir d'octets bruts, l'état logique que le noyau entretenait à l'instant de la capture.

> *Le disque est une archive. La mémoire est une scène de crime encore chaude — on la relève vite, ou on la perd.*

Adressage virtuel, pages et traduction d'adresses

CHAPITRE 02 · LE TERRAIN

Chaque processus croit disposer de tout l'espace d'adressage. Cette illusion — l'adressage virtuel — est maintenue par le noyau et le processeur au moyen de tables de pages. Comprendre cette mécanique, c'est comprendre pourquoi un outil comme Volatility doit « traduire » chaque adresse avant de lire quoi que ce soit.

2.1 Adresse virtuelle, adresse physique

Le code manipule des **adresses virtuelles** ; la RAM, elle, n'a que des **adresses physiques**. Entre les deux, le processeur effectue à chaque accès une traduction à l'aide de structures que le noyau prépare par processus. La mémoire est découpée en **pages** de 4 Kio (avec des grandes pages de 2 Mio ou 1 Gio pour certaines régions). Chaque page virtuelle est soit mappée vers une page physique (un *frame*), soit marquée non résidente — par exemple parce qu'elle a été écrite dans le *pagefile*.

2.2 Les tables de pages, étage par étage

En x86-64, la traduction traverse quatre niveaux de tables (PML4 → PDPT → PD → PT). L'adresse virtuelle est découpée en indices qui sélectionnent une entrée à chaque étage ; la dernière entrée pointe vers le *frame* physique, auquel on ajoute l'*offset* de page. Le point de départ de toute la chaîne est le registre `CR3` du processeur, qui contient le **Directory Table Base** (DTB) du processus courant : c'est l'ancre qui dit « voici l'espace d'adressage de *ce* processus ».

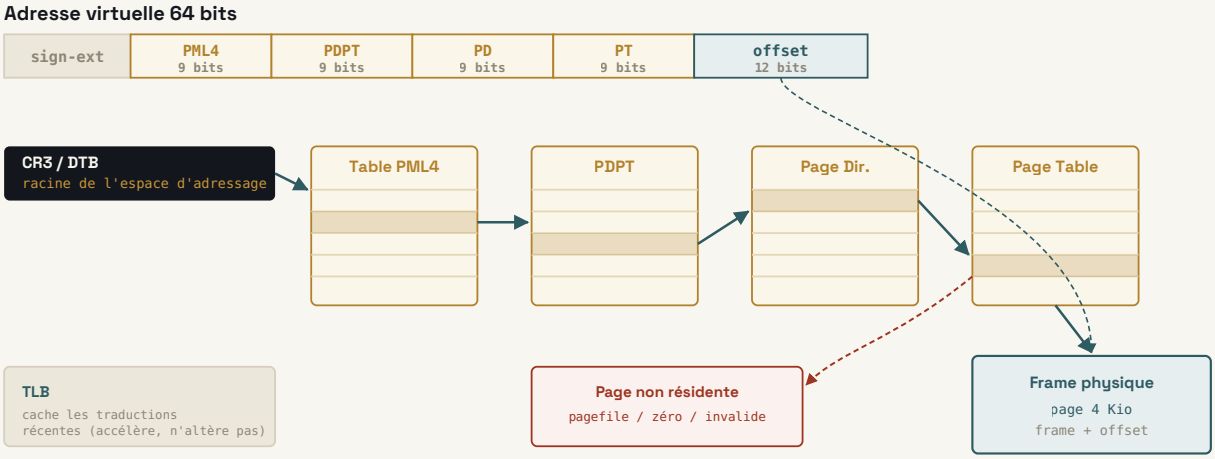


FIG. 2.1 — Traduction d'une adresse virtuelle 64 bits : `CR3/DTB` ancre la hiérarchie PML4 → PDPT → PD → PT, et l'offset désigne l'octet dans le frame. Une page peut être non résidente : c'est tout l'enjeu du pagefile et des « trous » dans une image mémoire.

2.3 Ce que cela change pour l'analyste

Pour lire la mémoire d'un processus donné, un framework doit d'abord retrouver son DTB, puis refaire exactement la traduction du processeur : c'est ce qui lui permet de passer d'une adresse virtuelle vue dans une structure à l'octet physique correspondant dans l'image. Trois conséquences concrètes en découlent.

Pages manquantes — Une page écrite dans le *pagefile* au moment de la capture est absente de l'image RAM. Sans le *pagefile*, certaines régions apparaissent vides – un *smear* légitime, pas forcément une dissimulation.

Espace noyau partagé — La moitié haute de l'espace d'adressage (le noyau) est partagée entre tous les processus ; la moitié basse est propre à chacun. Beaucoup d'artefacts d'intrusion vivent précisément à la frontière des deux.

Contiguïté trompeuse — Deux pages virtuellement voisines peuvent être physiquement éparpillées. On ne « scanne » pas une image comme un fichier linéaire : on la parcourt via les structures, ou on la balaie par signatures (*pool scanning*).

Le processus vu par le noyau

CHAPITRE 03 · LE TERRAIN

Un processus n'est pas une case dans une liste : c'est une constellation de structures que le noyau tient à jour. Savoir lesquelles, et comment elles se répondent, c'est disposer de la carte que suivent — et que trahissent — à la fois les plugins d'analyse et les techniques de dissimulation.

3.1 EPROCESS, la pièce maîtresse

Sous Windows, chaque processus est décrit par un objet noyau `_EPROCESS`. C'est le point d'entrée de presque tout le reste. Parmi ses champs, quelques-uns reviennent sans cesse en analyse :

`ImageFileName`	Nom court de l'exécutable (tronqué à 15 caractères) — pratique mais falsifiable ; ne jamais s'y fier seul.
`UniqueProcessId`	Le PID. Son parent est donné par `InheritedFromUniqueProcessId` (le PPID), base de l'arbre des processus.
`DirectoryTableBase`	Le DTB du processus : la clé qui permet de lire *son* espace d'adressage virtuel.
`ActiveProcessLinks`	Les liens avant/arrière (FLINK/BLINK) qui chaînent tous les processus actifs. C'est ce que parcourt **`pslist`** — et ce qu'un rootkit délie.
`VadRoot` · `Peb` · `ObjectTable` · `ThreadListHead`	Les ancres vers, respectivement, la carte mémoire du processus, son bloc utilisateur, sa table de handles et ses threads.

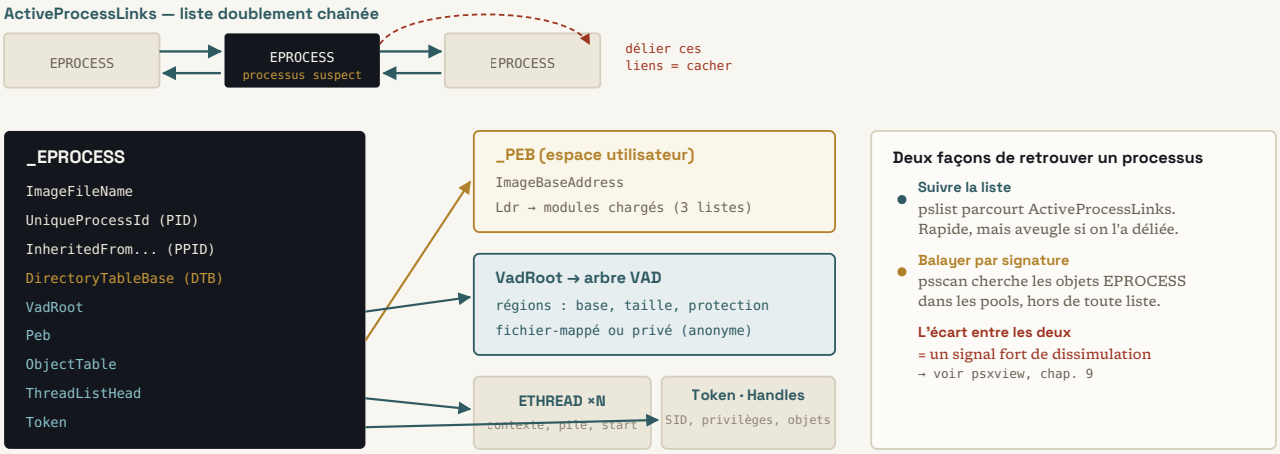


FIG. 3.1 — Anatomie d'un processus en mémoire. Depuis `_EPROCESS`, le noyau ancre le PEB (côté utilisateur), l'arbre VAD (carte mémoire), les threads, le token et la table de handles. La dualité suivre la liste / balayer les pools est au cœur de la détection des processus cachés.

3.2 Le PEB et les trois listes de modules

Côté utilisateur, le `_PEB` (Process Environment Block) porte l'adresse de base de l'image (`ImageBaseAddress`) et, via son champ `Ldr`, trois listes chaînées décrivant les modules chargés : ordre de chargement, ordre en mé-

moire, ordre d'initialisation. Ces trois listes *devraient* rester cohérentes. Un module présent dans l'une mais absent d'une autre est une anomalie classique : c'est exactement ce que traque `ldrmodules`. Le PEB étant en espace utilisateur, il est **modifiable par le processus lui-même** – donc un terrain privilégié de mascarade (voir `pebmasquerade`).

3.3 Le VAD : la carte mémoire qui ne ment pas facilement

L'arbre VAD (*Virtual Address Descriptor*) décrit chaque région de l'espace d'adressage d'un processus : plage, taille, protection (lecture/écriture/exécution) et nature – région **mappée** sur un fichier, ou région **privée** (anonyme). Cette distinction est de l'or pour l'analyste : du code exécutable dans une région privée, sans fichier de support, est fortement suspect. C'est le principe même de `malfind`. Le VAD est maintenu par le noyau pour la gestion mémoire ; le falsifier sans casser le processus est beaucoup plus difficile que de retoucher un champ du PEB.

> **À GARDER EN TÊTE**
>
> Les structures proches de l'utilisateur (PEB, ImageFileName) sont faciles à maquiller ; celles dont le noyau a besoin pour *fonctionner* (VAD, DTB, objets en pool) le sont beaucoup moins. En analyse, on privilégie les secondes comme points d'ancrage, et l'on traite les premières comme des indices à corroborer.

Le royaume noyau : pools, objets et surface DKOM

CHAPITRE 04 · LE TERRAIN

Au-dessus des processus, le noyau gère sa propre mémoire — les pools — où vivent les objets du système : processus, threads, fichiers, pilotes, connexions. C'est là que se jouent les manipulations les plus profondes, et là que le balayage par signatures révèle ce que les listes officielles taisent.

4.1 Pools noyau : paginé, non paginé, grandes allocations

Le noyau alloue ses structures dans des **pools**. Le pool *non paginé* (toujours résident en RAM, pour le code qui s'exécute à haut niveau d'interruption) et le pool *paginé* (susceptible d'être échangé) se partagent l'essentiel. Les allocations volumineuses passent par les *big pools*, suivis séparément (`bigpools`). Historiquement, les allocations portaient une étiquette de quatre caractères (*pool tag*) qui identifiait le type d'objet — un repère précieux pour le balayage, même si les systèmes récents en ont réduit la fiabilité.

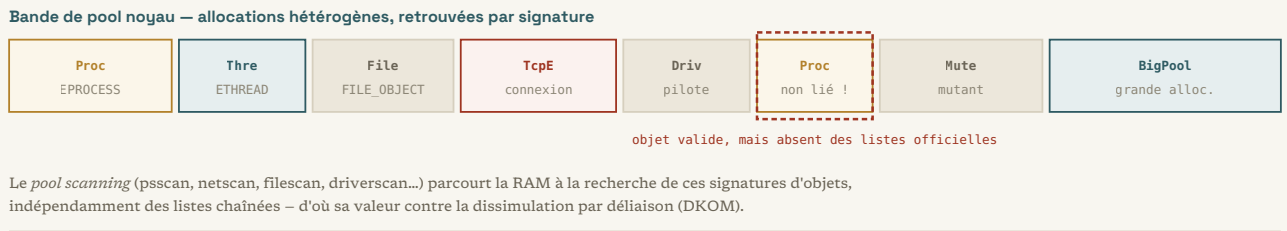


FIG. 4.1 — Le pool noyau mêle des objets de types variés. Le balayage par signature les retrouve même lorsqu'ils ont été retirés des listes que consultent les outils système.

4.2 DKOM : mentir au noyau en retouchant ses structures

La manipulation directe des objets du noyau — *Direct Kernel Object Manipulation* — consiste à modifier en mémoire les structures que le système tient pour vraies. L'exemple canonique : délier un `_EPROCESS` de la liste `ActiveProcessLinks`. Le processus continue de s'exécuter (le planificateur travaille à partir des threads, pas de cette liste), mais il disparaît des énumérations qui suivent les liens. D'autres variantes ciblent les listes de modules, de pilotes, de handles.

La parade est structurelle : on ne fait pas confiance à une seule source. On **croise plusieurs vues** — listes chaînées, balayage de pool, tables de sessions, tables de handles — et toute incohérence devient un indice. C'est la philosophie de `psxview`, qui confronte sept manières différentes d'énumérer les processus.

> LE FIL ROUGE DE LA PARTIE I
>
> Le noyau maintient des structures parce qu'il en a besoin pour fonctionner. L'attaquant, lui, cherche à mentir sur ces structures sans casser le système. L'analyste se place exactement dans cet interstice : **là où le mensonge doit rester partiel pour que la machine reste en vie**, une incohérence subsiste toujours quelque part. Tout le reste du guide consiste à savoir où la chercher.

PARTIE II

Acquérir proprement, lire la RAM

Une analyse ne vaut que ce que vaut sa capture. Cette partie couvre l'acquisition défendable — ordre de volatilité, outils, intégrité, pièges — puis ouvre le capot de Volatility 3 : couches de traduction, symboles, automagic. Comprendre ce que l'outil lit vraiment, c'est cesser de lui faire une confiance aveugle.

Acquisition : ordre de volatilité, outils, intégrité, pièges

CHAPITRE 05 · ACQUÉRIR & LIRE

Capturer la RAM d'une machine en marche est un acte intrusif et non atomique : la cible bouge pendant qu'on la lit. Le faire proprement, c'est accepter cette réalité et documenter chaque geste pour qu'il tienne devant un pair — ou un tribunal.

5.1 La règle d'or : la mémoire d'abord

Sur un hôte compromis mais toujours allumé, la mémoire se capture **avant** l'image disque et avant toute action d'isolation ou de remédiation. Couper le réseau, tuer un processus ou éteindre la machine détruit irrémédiablement une partie de l'état que l'on cherche à figer. Si et seulement si la mémoire ne peut pas être capturée et que l'extinction est imminente, l'imagerie disque reprend la priorité — mais c'est l'exception.

5.2 Outils d'acquisition

SÉLECTION D'OUTILS D'ACQUISITION MÉMOIRE (OPEN SOURCE & COURANTS EN DFIR)

OUTIL	PLATEFORME	NOTE D'USAGE
`WinPMEM`	Windows	Référence open source de longue date ; pilote noyau signé, sortie fichier ou flux. Souvent le premier réflexe.
`Magnet RAM Capture` · `DumpIt`	Windows	Acquisition simple et rapide, en un double-clic ; pratiques en réponse d'urgence.
`FTK Imager`	Windows	Capture la mémoire physique *et* le *pagefile*, en plus de l'imagerie disque.
`LiME`	Linux	Module noyau de référence ; produit une image au format `lime` (ou brut). Nécessite un module adapté au noyau cible.
`AVML`	Linux	Microsoft, en espace utilisateur (Rust, binaire statique) ; capture sans connaître a priori la distribution, compression Snappy optionnelle, format LiME.
`F-Response` · `Velociraptor` · `EDR`	À distance	Acquisition mémoire distante à l'échelle d'un parc ; de nombreux EDR intègrent une capture mémoire déclenchable à distance.

Les noms et disponibilités évoluent ; vérifiez la signature et la compatibilité noyau de votre outil avant toute intervention réelle.

5.3 Sources « gratuites » souvent oubliées

- **Instantanés de machines virtuelles.** Un fichier `.vmem` (VMware) est, de fait, une image mémoire déjà acquise, sans agent ni intrusion sur l'invité.
- **Fichier d'hibernation.** `hiberfil.sys` contient un instantané compressé de la RAM au moment de la mise en veille prolongée ; des outils savent le convertir pour l'analyser.
- **Pagefile / swap.** Complément indispensable d'une image RAM : il contient les pages qui en étaient absentes au moment de la capture.

5.4 Intégrité, et le piège du « smear »

Dès la capture terminée, on calcule une empreinte (SHA-256) de l'image et on la consigne : c'est la base de la chaîne de preuve. Mais une subtilité propre à la mémoire mérite d'être intégrée une fois pour toutes.

> **PIÈGE — L'INCOHÉRENCE N'EST PAS TOUJOURS UNE ATTAQUE**
>
> L'acquisition d'un système vivant n'est **pas atomique** : pendant que l'outil lit la RAM page par page, le système continue de modifier ces pages. Il en résulte un *smear* – des structures capturées à des instants légèrement différents, parfois incohérentes entre elles. Un pointeur qui « ne retombe pas juste », une liste partiellement mise à jour : avant de crier au rootkit, demandez-vous si vous ne regardez pas simplement les coutures de votre propre capture. On réduit le smear en capturant vite, au calme, et en évitant de charger la machine.

Dernier rappel qui justifie toute l'urgence : les **clés de chiffrement de volume** (BitLocker, VeraCrypt) ne vivent en clair qu'en RAM, tant que le volume est monté. Rater la capture mémoire, c'est parfois rendre le disque chiffré définitivement illisible.

Ce que lit réellement Volatility 3

CHAPITRE 06 · ACQUÉRIR & LIRE

Volatility 3 ne « regarde » pas une image mémoire comme un humain lit un texte. Il empile des couches de traduction, résout des structures à l'aide de tables de symboles, et reconstruit un modèle d'objets. Savoir ce qu'il lit — et ce qu'il choisit de ne plus lire — rend l'analyste lucide sur ses résultats.

6.1 Deux composants : des couches et des symboles

Tout l'état nécessaire à l'exécution d'un plugin est rassemblé dans un *contexte*, qui contient deux briques fondamentales : des **couches de données** et des **symboles** disponibles. Les anciens « espaces d'adressage » de Volatility 2 sont désormais des **couches de traduction** : chacune se pose sur une autre et traduit les adresses d'un niveau logique vers le niveau physique inférieur. Surtout, une couche peut avoir plusieurs dépendances — ce qui permet d'intégrer proprement le *swap* en l'empilant sous la couche virtuelle.

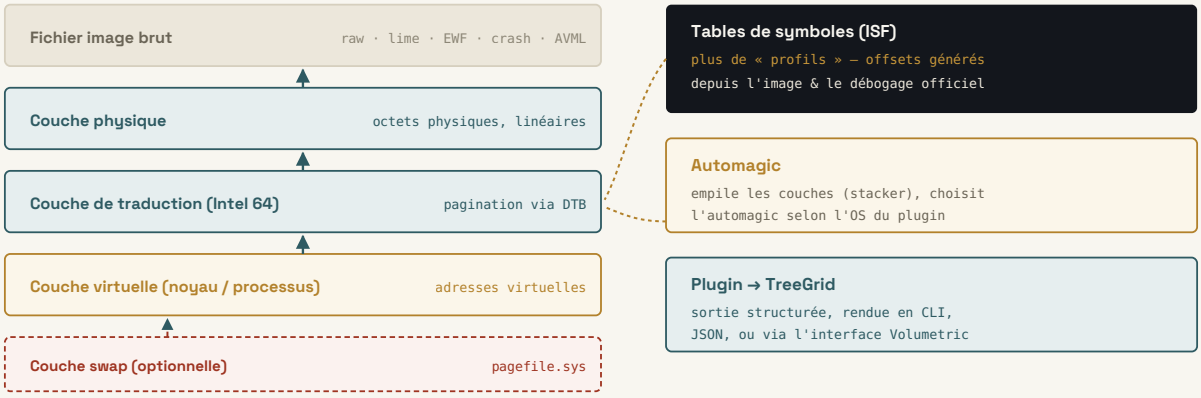


FIG. 6.1 — Chaîne de lecture de Volatility 3 : les couches de traduction transforment des octets bruts en adresses virtuelles exploitables ; les **`symboles ISF`** donnent le sens des structures ; l'**`automagic`** assemble le tout ; le plugin produit un **`TreeGrid`**.

6.2 Fini les profils : les symboles ISF

Changement majeur par rapport à Volatility 2 : **plus de profils**. Volatility 3 s'appuie sur une vaste bibliothèque de tables de symboles au format intermédiaire (ISF), et sait en *générer* pour la plupart des images Windows à partir de l'image elle-même et des informations de débogage officielles. Résultat : les offsets des structures sont connus avec précision pour la version exacte du système analysé, sans deviner un profil. C'est à la fois plus robuste et plus rapide — mais cela suppose un accès aux symboles correspondants, un point à anticiper en environnement hors-ligne.

6.3 Une lecture figée, par choix

Volatility 3 lit la donnée **une fois**, au moment où l'objet est construit, puis la considère stable. Ce choix assumé écarte l'analyse « en direct » où la donnée sous-jacente changerait pendant l'exécution — un usage marginal qui coûtait de nombreuses relectures. Pour l'analyste travaillant sur image, c'est transparent et bénéfique : on raisonne sur un instantané cohérent, rejouable.

6.4 Le socle : plugins d'inventaire et de triage

Avant toute chasse, on dresse l'état des lieux. Ces plugins forment le socle de presque toute analyse Windows.

PLUGINS SOCLE — INVENTAIRE & TRIAGE (PRÉFIXE `WINDOWS.`)

PLUGIN	CE QU'IL DONNE
`info`	Profil du système : version du noyau, architecture, DTB, horodatage – le point de départ.
`pslist · pstree`	Processus en suivant les listes du noyau ; `pstree` en restitue la filiation parent/enfant.
`psscan`	Processus retrouvés par balayage de pool, y compris terminés ou déliés.
`cmdline · envars`	Lignes de commande et variables d'environnement par processus – contexte d'exécution.
`dlllist · ldrmodules`	Modules chargés par processus ; `ldrmodules` confronte les trois listes du PEB.
`handles · getsids`	Objets ouverts (fichiers, clés, mutants…) et identités (SID) rattachées au processus.
`netscan · netstat`	Connexions et points d'écoute reconstruits depuis les objets réseau en mémoire.
`filescan · dumpfiles`	Objets fichier en mémoire, et extraction de leur contenu mis en cache.
`modules · modscan · svcscan`	Modules noyau (par liste et par balayage) et services enregistrés.
`hivelist · printkey`	Ruches de registre chargées en mémoire et lecture de clés précises.

Liste indicative, non exhaustive. Depuis l'image, Volatility 3 peut aussi reconstruire des artefacts disque mis en cache (MFT via ***`mftscan`****, etc.).*

PARTIE III

La chasse : détecter en mémoire

Injection, process hollowing, processus et connexions cachés, hooks, secrets en clair. Pour chaque technique, une même grille de lecture : l'idée, le mécanisme, ce que l'on voit en mémoire, le piège à éviter, et l'essentiel à retenir. On ne cherche pas « du mal » au hasard : on cherche des incohérences que l'attaquant n'a pas pu effacer sans casser la machine.

Une grille de lecture commune

PARTIE III · MODE D'EMPLOI

Les six chapitres qui suivent partagent une même structure. L'objectif n'est pas de mémoriser des commandes, mais d'ancrer un réflexe : relier chaque technique d'attaque à la trace qu'elle laisse — ou ne peut pas ne pas laisser — en mémoire.

Idée clé	En une phrase, ce que l'attaquant cherche à obtenir.
Mécanisme	Comment il s'y prend, au niveau des structures et des API — juste assez pour comprendre la trace.
Ce qu'on voit en mémoire	Les artefacts concrets et les plugins qui les révèlent. C'est le cœur opérationnel.
Piège & anti-forensique	Faux positifs courants et contre-mesures de l'adversaire : ce qui peut vous tromper dans un sens comme dans l'autre.
À retenir	L'essentiel compressé, pour la relecture rapide.

> *Une image mémoire n'avoue pas ; elle se contredit. Le travail de l'analyste est de faire parler les contradictions.*

Injection de code en mémoire

CHAPITRE 07 · LA CHASSE

Faire exécuter son code par un autre processus — légitime, de confiance — pour hériter de sa réputation, de ses droits et de sa discrétion. L'injection est la brique de base de presque toute l'activité « in-memory ».

FICHE TECHNIQUE · T1055

Injection de code

IDÉE CLÉ

Placer du code exécutable dans l'espace d'un processus hôte et l'y faire tourner, sans écrire de fichier sur disque.

MÉCANISME

Le schéma classique alloue une région mémoire dans le processus cible (souvent en lecture-écriture-exécution), y écrit la charge, puis lance un thread ou détourne un thread existant vers cette région. Les variantes modernes raffinent chaque étape pour éviter l'allocation RWX visible, mais la logique reste : du code tournant depuis une région qui n'a pas de fichier derrière elle.

CE QU'ON VOIT EN MÉMOIRE

Une région **VAD privée** (non mappée sur un fichier) portant une protection **exécutable**, contenant du code — fréquemment un en-tête PE, parfois volontairement effacé. `malfind` cible précisément ce motif : il liste les plages mémoire susceptibles de contenir du code injecté, en s'appuyant sur la nature privée de la VAD et sa protection. On corrobore avec `suspicious_threads` (threads démarrant dans une région anormale) et un *dump* de la région pour analyse.

PIÈGE & ANTI-FORENSIQUE

Des moteurs JIT (navigateurs, .NET, Java) génèrent légitimement du code exécutable en mémoire privée : malfind produit des faux positifs qu'il faut savoir écarter par le contexte (quel processus, quelle région, quel contenu). À l'inverse, l'attaquant efface l'en-tête PE, réutilise une région déjà exécutable, ou passe par des techniques sans allocation RWX pour rester sous le radar. Un résultat vide de malfind ne prouve rien ; un résultat plein n'est pas une condamnation.

À RETENIR

Du code exécutable dans une VAD privée sans fichier de support est le signal d'injection le plus robuste. malfind le met en lumière, mais c'est le *contexte* — processus, thread, contenu — qui tranche entre JIT bénin et charge hostile.

Process hollowing (processus creux)

CHAPITRE 08 · LA CHASSE

Vider un processus légitime de sa substance pour le remplir de code hostile, tout en gardant son nom, son chemin et son identité. Le résultat : un `svchost.exe` qui n'est plus `svchost.exe` — mais qui en a toute l'apparence.

FICHE TECHNIQUE · T1055.012

Process hollowing

IDÉE CLÉ

Lancer un processus de confiance à l'état suspendu, remplacer son image en mémoire par une image malveillante, puis le relancer. Le processus garde une identité légitime ; son code ne l'est plus.

MÉCANISME

La séquence est remarquablement stéréotypée (voir FIG. 8.1) : création suspendue, lecture du PEB pour localiser la base de l'image, « creusement » de la section d'origine, allocation à la même base, écriture de l'image hostile, réécriture du point d'entrée puis reprise du thread.

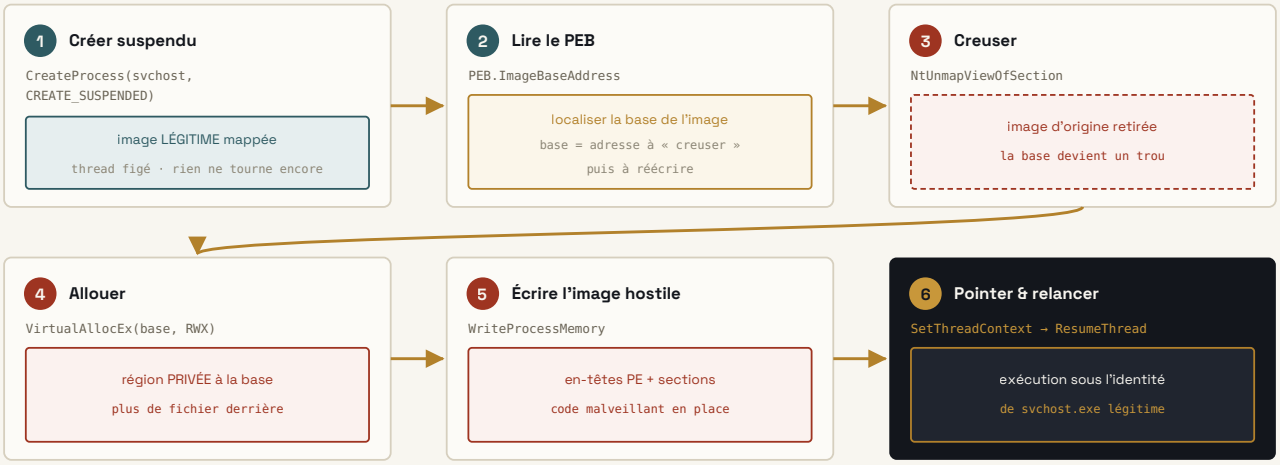


FIG. 8.1 — Process hollowing en six temps. Les étapes 3 à 5 (rouge) rompent le lien entre le nom du processus et son contenu : à la fin, l'image à la base n'a plus de fichier de support correspondant — l'empreinte même que recherche `hollowprocesses`.

CE QU'ON VOIT EN MÉMOIRE

Une **discordance entre l'identité et le contenu** : le chemin et le nom disent `svchost.exe`, mais l'image mappée à la base ne correspond plus au fichier sur disque, ou la région à la base est devenue privée au lieu d'être mappée sur l'exécutable. **`hollowprocesses`** est dédié à ce motif. On recoupe avec **`ldrmodules`** (incohérence des listes du PEB), **`malfind`** (région exécutable privée à la base), et un contrôle de vraisemblance parent/enfant via **`pstree`** (un `svchost` dont le parent n'est pas `services.exe` interpelle).

PIÈGE & ANTI-FORENSIQUE

Certaines protections légitimes (réécriture de code, *relocations*, mises à jour à chaud) peuvent créer des écarts bénins entre image mappée et fichier disque. À l'inverse, les variantes avancées (*process doppelgänging*, *ghosting*, *herpaderping*) jouent sur l'ordre des opérations et les sections transactionnelles pour que l'image semble légitime au moment de l'inspection. Ne jugez jamais sur un seul indicateur : c'est la **convergence** nom/parent/contenu/listes qui fait la preuve.

À RETENIR

Le hollowing casse le lien « nom légitime ↔ contenu légitime ». Cherchez la discordance : image mappée qui ne colle pas au fichier, région privée à la base, filiation improbable. `hollowprocesses` en tête, corroboré par ldrmodules, malfind et pstree.

Processus et modules cachés

CHAPITRE 09 · LA CHASSE

Ce qui n'apparaît dans aucune liste n'a pas disparu : il a été délié. La détection des objets cachés ne repose pas sur une astuce, mais sur un principe — comparer des énumérations obtenues par des chemins indépendants, et traiter tout écart comme un aveu.

FICHE TECHNIQUE · T1014 / DKOM

Processus & modules cachés

IDÉE CLÉ

Retirer un objet (processus, module, pilote) des listes que consultent les outils système, sans l'empêcher de fonctionner — pour le soustraire au regard tout en le gardant actif.

MÉCANISME

La dissimulation la plus répandue délie l'objet de sa liste chaînée (DKOM) : un `_EPROCESS` retiré de `ActiveProcessLinks` reste planifié via ses threads, mais échappe à **`pslist`**. Les mêmes idées valent pour les modules noyau (listes de chargement) et les pilotes.

CE QU'ON VOIT EN MÉMOIRE

L'objet subsiste dans les **pools** et reste trouvable par balayage de signature. **`psscan`** voit les processus que **`pslist`** ne voit plus ; l'écart entre les deux est le signal. **`psxview`** systématise l'approche en confrontant jusqu'à sept vues distinctes (listes, balayage, sessions, tables de handles…) : une case « absent » isolée pointe la dissimulation. Côté noyau, on oppose **`modules`** (par liste) à **`modscan`** (par balayage), et **`driverscan`** pour les pilotes.

PIÈGE & ANTI-FORENSIQUE

Un processus légitimement terminé juste avant la capture apparaît lui aussi « seulement au scan » — son objet survit un temps dans le pool sans être dans la liste des actifs : ce n'est pas une dissimulation. À l'inverse, un adversaire sophistiqué peut corrompre les signatures de pool ou réutiliser des structures pour brouiller le balayage. Datez vos anomalies (horodatages de création/sortie) avant de conclure.

À RETENIR

La détection des cachés = croisement de vues indépendantes. `psxview` et l'écart `pslist`/`psscan` en tête ; `modscan`/`driverscan` côté noyau. Toujours dater l'anomalie pour écarter le processus simplement terminé.

Connexions et artefacts réseau volatils

CHAPITRE 10 · LA CHASSE

L'état réseau est parmi les preuves les plus fugaces : une connexion de commande-et-contrôle peut n'exister que quelques secondes. La RAM en garde la trace là où aucun journal ne l'a consignée — à condition de la capturer à temps, et de la lire avec prudence.

FICHE TECHNIQUE · T1049 / C2

Artefacts réseau en mémoire

IDÉE CLÉ

Reconstruire, depuis les objets réseau du noyau, qui parlait à qui et depuis quel processus — y compris pour des connexions déjà fermées ou volontairement masquées.

MÉCANISME

Le noyau matérialise connexions et points d'écoute par des objets en pool (structures d'endpoints TCP/UDP). Même après fermeture, ces structures peuvent subsister un temps avant recyclage — autant d'empreintes exploitables.

CE QU'ON VOIT EN MÉMOIRE

`netscan` balaie les pools à la recherche de ces objets et restitue adresses, ports, état et **processus propriétaire** — y compris des connexions absentes des outils en ligne. `netstat` adopte une approche complémentaire par suivi de structures. On relie la connexion au processus via le PID, puis on recoupe avec `cmdline`, les modules chargés et les régions suspectes (malfind) du même processus. Les traces résiduelles d'un *beaconing* régulier (mêmes couples IP/port réapparaissant) orientent vers un canal C2.

PIÈGE & ANTI-FORENSIQUE

Une image mémoire est un instantané : une connexion peut y figurer comme active alors qu'elle venait de se fermer, ou l'inverse. Les structures partiellement recyclées produisent des entrées fantômes ou incohérentes — à ne pas surinterpréter. Enfin, un attaquant discret limite les connexions persistantes (trafic court, à la demande) : l'absence de C2 flagrant ne vaut pas absence de compromission. Corroborez avec le contexte processus.

À RETENIR

La mémoire capture le réseau que les journaux ratent. `netscan` relie connexion ↔ processus ; on remonte ensuite au code. Attention à l'instantané : état « actif/fermé » et entrées fantômes se lisent avec réserve.

Hooks : détourner le flux et aveugler l'EDR

CHAPITRE 11 · LA CHASSE

Un hook intercale du code adverse sur le chemin d'une fonction. Côté offensif, c'est le moyen d'espionner, de filtrer ou de mentir — y compris pour désactiver la télémétrie de sécurité. Côté défensif, chaque hook est une déviation mesurable par rapport à un état de référence connu.



FIG. 11.1 — Un hook insère du code adverse sur le chemin d'une fonction. La famille « évasion EDR » inverse la logique : elle retire ou contourne les hooks légitimes de la solution de sécurité, laissant sa propre signature en mémoire.

FICHE TECHNIQUE · T1562 / T1574

Hooks & neutralisation de la télémétrie

IDÉE CLÉ

Prendre le contrôle du flux d'exécution – ou, à l'inverse, défaire la surveillance – en modifiant des pointeurs, des tables ou les premiers octets de fonctions clés.

MÉCANISME — LES GRANDES FAMILLES

On distingue surtout : les **hooks de table** (SSDT côté noyau, IAT/EAT côté PE), qui réécrivent une entrée de pointeur ; les **hooks inline**, qui remplacent les premiers octets d'une fonction par un saut ; les **callbacks et IRP** détournés côté noyau ; et la famille **évasion de l'EDR** – *patch* d'ETW, désamorçage des hooks de `ntdll`, syscalls directs ou indirects pour court-circuiter la télémétrie.

CE QU'ON VOIT EN MÉMOIRE

Chaque famille a son plugin. **`ssdt`** révèle les entrées de la table des appels système pointant hors des modules attendus ; **`iat`** fait de même pour les imports d'un PE. **`callbacks`** liste les routines de notification du noyau, **`driverirp`** les gestionnaires d'IRP déviés. Pour l'évasion moderne : **`etwpatch`** détecte l'altération des fonctions ETW ; **`unhooked_system_calls`**, **`direct_system_calls`** et **`indirect_system_calls`** repèrent les appels système qui contournent les hooks de l'EDR ; **`debugregisters`** débusque les hooks par registres de débogage matériels. Côté noyau, **`drivermodule`** et **`svcdiff`** confrontent l'état observé aux listes attendues ; **`skeleton_key_check`** cible une altération spécifique de l'authentification.

PIÈGE & ANTI-FORENSIQUE

Les solutions de sécurité *elles-mêmes* posent des hooks légitimes (AV, EDR, outils d'instrumentation) : la présence d'un hook n'est pas un verdict. Tout est dans la **cible** et l'**origine** : un hook pointant vers un module non signé, une région privée ou un pilote inconnu change de nature. À l'inverse, un attaquant peut retirer ses hooks juste après usage – d'où l'intérêt d'une capture au bon moment et du croisement avec les autres indicateurs.

À RETENIR

Un hook est une déviation par rapport à une référence. Identifiez la famille (SSDT/IAT/inline/callbacks) et, pour l'évasion EDR, pensez `etwpatch` et la triade `* system calls`. Jugez par la cible du détournement, pas par sa simple existence.

Identifiants et clés en clair

CHAPITRE 12 · LA CHASSE

Pour qu'un système authentifie, chiffre ou déchiffre, il lui faut à un moment le secret en clair en mémoire. C'est une fatalité cryptographique — et une opportunité d'enquête. La RAM est le seul endroit où ces secrets existent lisiblement.

FICHE TECHNIQUE · T1003 / T1552

Secrets, hachages & clés

IDÉE CLÉ

Récupérer, depuis la mémoire, les éléments d'authentification et les clés cryptographiques — soit pour comprendre ce qu'un attaquant a pu voler, soit pour poursuivre l'investigation (déchiffrer un volume, pivoter légitimement).

MÉCANISME

Windows conserve des éléments d'authentification dans des structures et des ruches de registre chargées en mémoire (SAM, SECURITY), protégés par des clés dérivées elles aussi présentes en RAM. Les clés de session, le matériel cryptographique applicatif et les clés de chiffrement de volume vivent dans l'espace des processus concernés tant qu'ils tournent.

CE QU'ON VOIT EN MÉMOIRE

`hashdump` reconstruit les hachages de comptes locaux depuis les ruches SAM/SYSTEM en mémoire ; `lsadump` extrait les secrets LSA ; `cachedump` récupère les identifiants de domaine mis en cache. Au-delà, un balayage de chaînes et de motifs dans l'espace d'un processus peut révéler mots de passe, *tokens* et clés applicatives en clair. Les clés de volume (BitLocker, VeraCrypt) ne sont lisibles que là, tant que le volume est monté — d'où, à nouveau, l'urgence de la capture.

PIÈGE & ANTI-FORENSIQUE

Ces éléments sont extrêmement sensibles : leur manipulation relève d'un cadre légal strict et d'une chaîne de preuve maîtrisée. Techniquement, les versions récentes de Windows durcissent la protection des secrets (isolation, chiffrement en mémoire), ce qui réduit ou transforme ce qui est lisible ; l'absence de hachages exploitables n'indique pas l'absence de compromission. Et comme toujours, le *smear* peut corrompre une structure et fausser une extraction.

À RETENIR

Tout secret utilisé transite en clair par la RAM. `hashdump`, `lsadump`, `cachedump` pour l'authentification ; balayage ciblé pour les clés applicatives et de volume. Manipulation sous cadre légal et chaîne de preuve rigoureuse.

12.1 Tableau récapitulatif — plugins de détection

PLUGINS ORIENTÉS DÉTECTION (WINDOWS) — PAR FAMILLE D'ACTIVITÉ

FAMILLE	PLUGINS	CE QU'ILS RÉVÈLENT
Injection	`malfind` · `suspicious_threads`	Code exécutable en région privée ; threads démarrant dans une zone anormale.

FAMILLE	PLUGINS	CE QU'ILS RÉVÈLENT
Hollowing & mascarade	`hollowprocesses` · `ldrmodules` · `pebmasquerade` · `processghosting`	Discordance identité/contenu ; listes PEB incohérentes ; images sans fichier de support.
Objets cachés	`psxview` · `psscan` · `modscan` · `driverscan`	Écarts entre énumérations ; objets présents au balayage mais hors des listes.
Réseau	`netscan` · `netstat`	Connexions et écoutes reliées au processus, y compris fermées ou masquées.
Hooks	`ssdt` · `iat` · `callbacks` · `driverirp` · `drivermodule`	Pointeurs et tables déviés vers des cibles inattendues.
Évasion EDR	`etwpatch` · `unhooked_system_calls` · `direct_system_calls` · `indirect_system_calls` · `debugregisters`	Télémétrie neutralisée ; appels système contournant les hooks ; hooks matériels.
Services & auth.	`svcdiff` · `skeleton_key_check`	Services incohérents ; altération ciblée de l'authentification.
Secrets	`hashdump` · `lsadump` · `cachedump`	Hachages locaux, secrets LSA, identifiants de domaine en cache.

NOTE DE VERSION

Dans Volatility 3, nombre de ces plugins de chasse sont regroupés sous le sous-paquet `windows.malware` (par ex. `windows.malware.hollowprocesses`). L'ancien alias de premier niveau `windows.malfind` est déprécié au profit de `windows.malware.malfind` . Les intitulés et la disponibilité dépendent de votre version : vérifiez-la sur votre déploiement.

PARTIE IV

Mettre la mémoire à sa juste place



La forensique mémoire n'est pas une discipline isolée. Elle prend tout son sens articulée à l'analyse de binaire, dans une méthode qui assume ses limites et sait reconnaître — sans paranoïa — la main de l'anti-forensique.

Mémoire vive & analyse de binaire : deux moitiés d'une enquête

CHAPITRE 13 · PERSPECTIVE

On oppose parfois « forensique mémoire » et « reverse engineering ». C'est un contresens. Les deux répondent à des questions différentes et se nourrissent l'une l'autre : l'une dit ce qui s'est passé ici, l'autre ce dont c'est capable.

13.1 Deux questions, deux preuves

ANALYSE DE BINAIRE

Part d'un **échantillon** et en déduit une **capacité**. Statique (désassemblage, structure du PE, imports) puis dynamique (exécution contrôlée) : que *peut* faire ce fichier ? Quelles fonctions, quels indicateurs, quelle logique ? Profondeur maximale, mais il faut déjà tenir le fichier – et savoir qu'il a compté.

FORENSIQUE MÉMOIRE

Part d'un **hôte** et en déduit un **fait**. Qu'est-ce qui s'est *réellement* exécuté, dans quel processus, avec quelles connexions, quels secrets déchiffrés ? Vérité de terrain sur un instant donné – y compris pour du code qui n'a jamais été un fichier. Largeur et contexte, moins de profondeur sur un artefact isolé.

13.2 Pourquoi la mémoire comble les angles morts du binaire

- **Le sans-fichier.** Charge purement en mémoire, injection réflective, .NET à la volée : l'analyse de binaire n'a rien à se mettre sous la dent tant que la mémoire n'a pas *extrait* l'artefact.
- **L'empaqueté et le chiffré.** Sur disque, l'échantillon est opaque. En mémoire, il est déballé pour s'exécuter : la RAM livre la forme « vraie » du code que le statique seul ne verrait jamais sans le lancer.
- **Le contexte d'exécution.** Configuration déchiffrée, clés de session, domaine C2 résolu, processus parent réel : autant d'éléments que le binaire, hors contexte, ne révèle pas.

13.3 La boucle vertueuse

En incident réel, les deux disciplines fonctionnent en boucle. La **mémoire trie et extrait** : malfind isole une région injectée, `dumpfiles` ou un *dump* de VAD en sort le code, netscan donne le C2. Ces artefacts **alimentent l'analyse de binaire**, qui en tire des indicateurs durables (motifs de hooks, structures de configuration, algorithmes). Ces indicateurs **réorientent la chasse mémoire** : on sait désormais quoi chercher sur les autres hôtes du parc. Et l'on recommence.

FORMULE À RETENIR

L'analyse de binaire donne la **profondeur** ; la forensique mémoire donne la **vérité de terrain** et le **contexte**. Opposer les deux, c'est se priver d'une moitié de l'enquête. Les grands dossiers se résolvent à la jonction.

Méthode, limites et anti-forensique

CHAPITRE 14 · PERSPECTIVE

Un bon analyste mémoire n'est pas celui qui connaît le plus de plugins, mais celui qui doute au bon endroit. Voici la méthode qui sous-tend tout ce guide, ses limites assumées, et la bonne posture face à l'anti-forensique.

14.1 Une méthode en quatre temps

1. **Établir la vérité de base.** `windows.info`, inventaire des processus, des modules, du réseau. On cartographie le normal avant de chercher l'anormal.
2. **Chercher les incohérences, pas « le mal ».** Croiser les vues (listes vs balayage), confronter identité et contenu, dater les anomalies. Le signal, c'est la contradiction.
3. **Corroborer avant de conclure.** Aucun indicateur isolé ne fait preuve : on exige la convergence de plusieurs sources indépendantes.
4. **Extraire et passer la main.** Sortir les artefacts (régions, modules, configs) et les confier à l'analyse de binaire ; réinjecter les indicateurs obtenus dans la chasse.

14.2 Limites à garder en tête

`Instantané non atomique`	Le *smear* crée des incohérences légitimes ; toute anomalie n'est pas une attaque.
`Pages manquantes`	Sans le *pagefile*, des régions sont absentes. Vide ≠ propre.
`Dépendance aux symboles`	Sans symboles adaptés à la version exacte, certaines structures restent illisibles – à anticiper hors-ligne.
`Durcissement de l'OS`	Les protections récentes réduisent ce qui est lisible (secrets, isolation) : l'absence de trace n'est pas une preuve d'innocence.

14.3 L'anti-forensique, sans paranoïa

L'adversaire sait que la mémoire sera analysée. Il efface des en-têtes PE, retire ses hooks après usage, limite les connexions persistantes, exploite le durcissement pour se fondre dans le bruit. La bonne réponse n'est ni la naïveté (« tout est propre ») ni la paranoïa (« tout est compromis »), mais la **discipline probatoire** : exiger la convergence, dater les faits, documenter chaque geste. C'est précisément parce que le mensoge doit rester *partiel* – pour que la machine continue de tourner – qu'il laisse toujours une couture visible quelque part.

> *On ne prouve pas une intrusion avec un plugin. On la prouve avec une histoire cohérente que plusieurs sources indépendantes racontent de la même façon.*

Aide-mémoire des plugins & sources vérifiées

CHAPITRE 15 · PERSPECTIVE

De quoi garder l'essentiel à portée de main : un fil d'analyse condensé, puis les sources à jour sur lesquelles s'appuie ce guide.

15.1 Fil d'analyse express

1 · Cadre. `windows.info` — version, architecture, DTB, horodatage.

2 · Processus. `pstree`, `pslist`, puis `psscan` et `psxview` pour les cachés ; `cmdline` pour le contexte.

3 · Injection. `malware.malfind` + `suspicious_threads` ; dump des régions suspectes.

4 · Mascarade. `hollowprocesses`, `ldrmodules`, `pebmasquerade` ; filiation via `pstree`.

5 · Réseau. `netscan` / `netstat` ; relier au PID, remonter au code.

6 · Noyau & hooks. `modscan`, `driverscan`, `ssdt`, `callbacks`, `driverirp`.

7 · Évasion EDR. `etwpatch`, `unhooked_system_calls`, `direct_system_calls`, `indirect_system_calls`, `debugregisters`.

8 · Secrets. `hashdump`, `lsadump`, `cachedump` — sous cadre légal.

9 · Extraction. `dumpfiles`, dump de VAD ; passer la main à l'analyse de binaire.

0 · Toujours. hash de l'image, horodatage des anomalies, corroboration multi-sources.

15.2 Sources vérifiées

Références consultées et vérifiées lors de la rédaction (octobre 2026). La documentation de Volatility 3 citée correspond à la série stable v2.28.

1 **Volatility 3 – Documentation officielle (stable).** Volatility Foundation.
https://volatility3.readthedocs.io/en/stable/

2 **Volatility 3 – Plugins Windows & sous-paquet** `windows.malware` (malfind, hollowprocesses, psxview, ldrmodules, pebmasquerade, processghosting, direct/indirect/unhooked_system_calls, etwpatch, debugregisters, svcdiff, skeleton_key_check…).
https://volatility3.readthedocs.io/en/stable/volatility3.plugins.windows.html
https://volatility3.readthedocs.io/en/stable/volatility3.plugins.windows.malware.html

3 **Changements entre Volatility 2 et 3** – couches de traduction, symboles ISF (fin des profils), automagic, lecture figée.
https://volatility3.readthedocs.io/en/stable/vol2to3.html

4 **Volatility 3 – Bases & glossaire** (contexte, couches, symboles, TreeGrid).
https://volatility3.readthedocs.io/en/stable/basics.html

5 **RFC 3227 – Guidelines for Evidence Collection and Archiving** (ordre de volatilité).
https://www.rfc-editor.org/rfc/rfc3227

6 **AVML – Acquire Volatile Memory for Linux.** Microsoft.
https://github.com/microsoft/avml

7 **MemProcFS – The Memory Process File System.** Ulf Frisk (LeechCore).
https://github.com/ufrisk/MemProcFS

8 **Awesome Memory Forensics** – panorama d'outils d'acquisition et d'analyse (WinPMEM, LiME, FTK Imager…).
https://github.com/digitalisx/awesome-memory-forensics

9 **The Art of Memory Forensics** – Ligh, Case, Levy, Walters (Wiley). Référence fondatrice de la discipline.

10 **SANS DFIR** – filières FOR508 / FOR532 (memory forensics & réponse à incident) ; posters et *cheat sheets* publics.
https://www.sans.org/

Les noms de plugins, formats et disponibilités évoluent au fil des versions de Volatility 3. Avant toute opération réelle, confirmez-les sur votre déploiement et vérifiez la compatibilité des symboles avec la version exacte du système analysé.

— 0xKuraishY · FORENSIQUE DE LA MÉMOIRE VIVE · ÉD. 2026